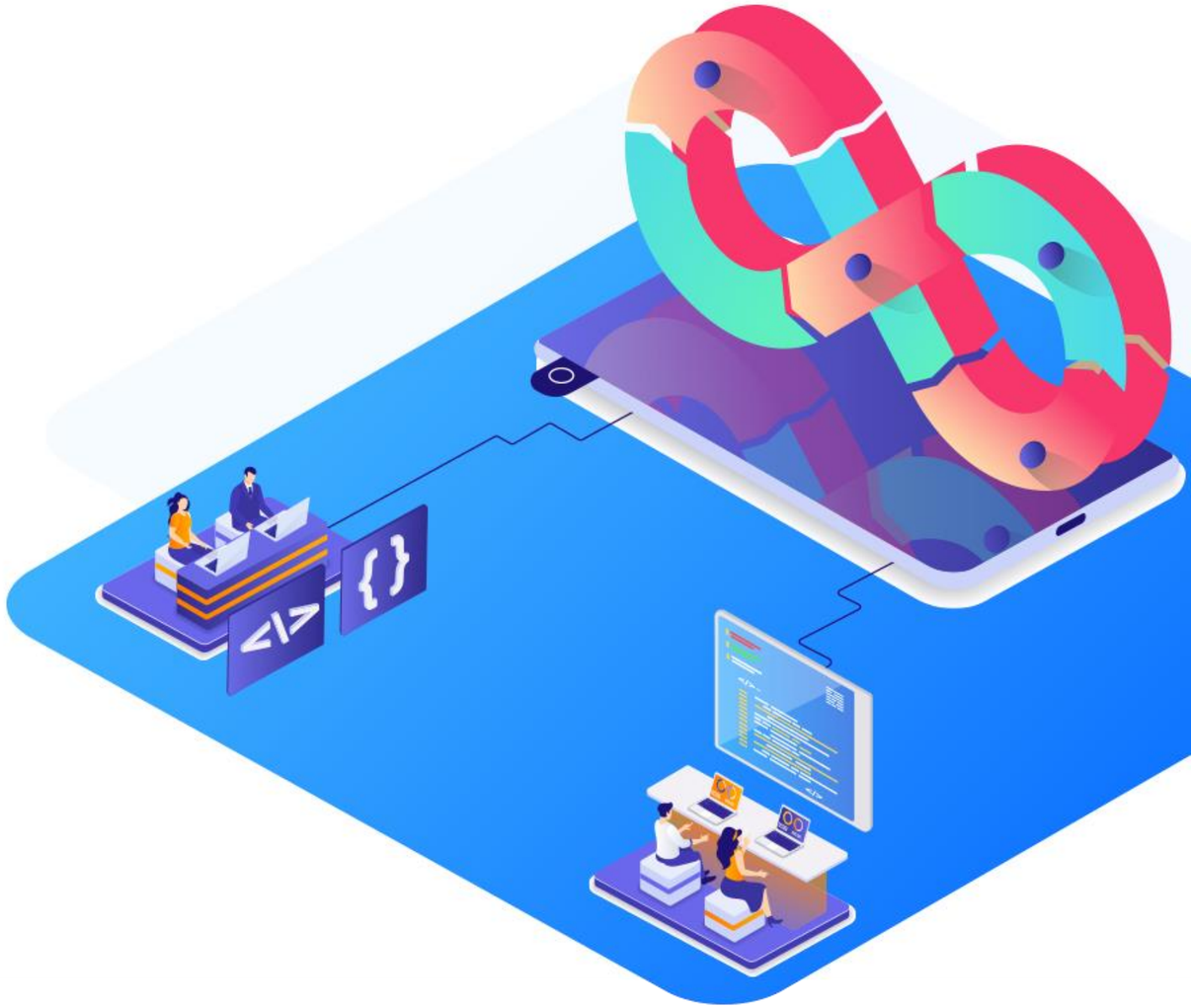
Capstone Project



Application Deployment to Multi-Cloud

Objective

The objective of this project is to showcase the end-to-end process of deploying a Spring Boot microservice application on multi-cloud environments (AWS and Azure) using a Jenkins pipeline, Ansible, and Docker. This project demonstrates the automation of CI/CD workflows, efficient containerization of the application, and seamless multi-cloud deployment, ensuring high availability and scalability across both cloud platforms.

Problem Statement and Motivation

Real-time scenario:

BankPro, a leading financial services company, operates a multi-cloud infrastructure to deliver its banking and payment services to a diverse client base across the US and Europe. To enhance the reliability and scalability of its core banking application, the company is adopting a containerized microservices architecture and deploying it across both AWS and Azure cloud platforms.

As a DevOps Engineer at BankPro, you are tasked with automating the CI/CD pipeline for the core banking microservice using Jenkins, Ansible, and Docker. The project involves setting up an efficient multi-cloud deployment strategy, ensuring seamless integration between AWS and Azure VMs, and enabling continuous delivery of the application updates. Your role is to implement a robust solution that automates the build, test, and deployment processes, reducing downtime and improving service availability for BankPro's customers.

Industry Relevance

The following tools used in this project serve specific purposes within the industry:

1. **Jenkins:** Jenkins is an open-source automation server used to automate the building, testing, and deployment of applications.

2. **AWS Cloud:** AWS Cloud provides scalable and flexible cloud services, including Amazon EC2 for launching virtual machines. It is used as one of the target environments for deploying the containerized application

3. **Azure Cloud:** Azure Cloud is Microsoft's cloud computing platform offering a wide range of services, including virtual machines. It serves as the second cloud platform for deploying the application, ensuring a multi-cloud deployment strategy.

4. **GitHub:** GitHub is a web-based platform for version control and collaboration, allowing teams to manage code repositories and track changes. It serves as the source code repository for the Jenkins pipeline in this project

Business Impact

Jenkins:

Jenkins has experienced a 50% increase in usage among enterprises seeking to streamline their CI/CD processes, particularly in multi-cloud and hybrid cloud environments.

The adoption growth reflects the industry's push towards automation, allowing businesses to accelerate software delivery cycles and reduce manual intervention in deployment workflows.

GitHub:

GitHub usage has surged by 45% among development teams implementing CI/CD workflows with Jenkins, emphasizing its role in collaborative software development and secure code management.

The growth signifies the industry's move towards centralized code repositories for improved version control, streamlined collaboration, and enhanced code quality.

Tasks

The following tasks outline the process of implementing a multi-cloud deployment pipeline for a Spring Boot microservice using Jenkins

1. Created a new GitHub repository and cloned it to the local machine to establish the development environment for the Spring Boot application.

2. Set up the Jenkins environment on the DevOps lab by accessing the pre-installed Jenkins instance, and created a new Jenkins job for automating the CI pipeline.

3. Forked the GitHub repository for the Spring Boot application and configured the Jenkins pipeline script.

4. Developed a Dockerfile for containerizing the Java application, then integrated Docker image build and publish stages within the Jenkins pipeline to automate the creation and push of Docker images to DockerHub

Tasks

The following tasks outline the process of implementing a multi-cloud deployment pipeline for a Spring Boot microservice using Jenkins

5. Configured Ansible on the Simplilearn lab to connect to AWS and Azure VMs, setting up inventory files and SSH key pairs for secure connectivity.

6. Launched VMs on both AWS and Azure using their respective consoles, configured security groups, and opened necessary ports for application access.

7. Executed the Jenkins pipeline to build, test, and deploy the application using Ansible playbooks, which automated the deployment of Docker containers across AWS and Azure VMs.

8. Conducted tests to validate the deployment by accessing the application through Swagger API on both cloud platforms, ensuring the setup was functional and met the expected performance criteria.

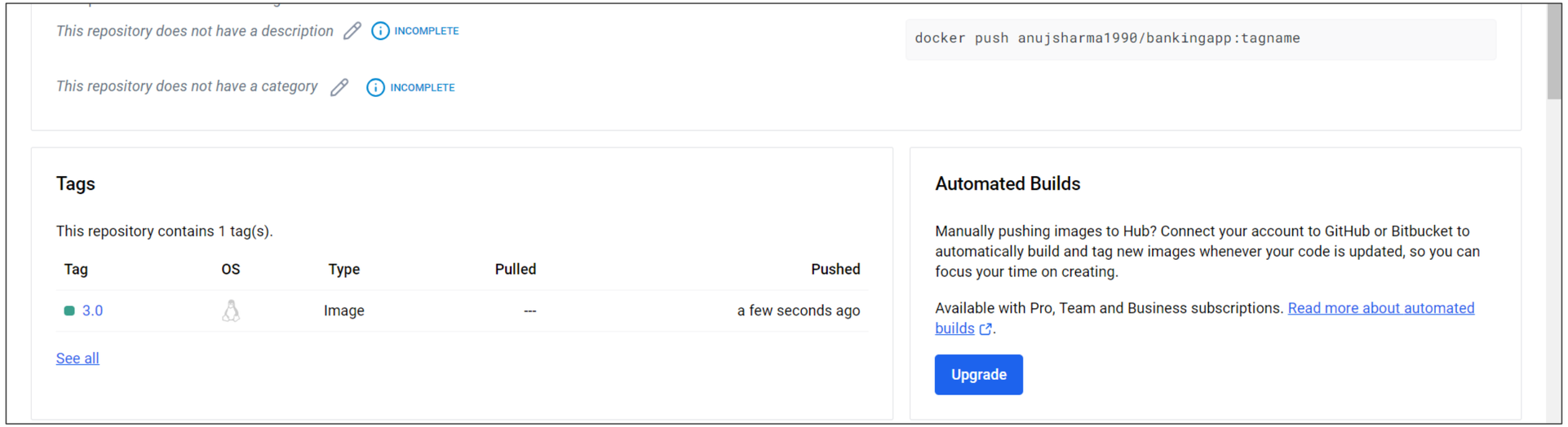
Project References

- **Task 1: Refer to the course:** DevOps Foundations-Version Control and CICD with Jenkins: Lesson 03
- **Task 2: Refer to the course:** DevOps on AWS: Lesson 02
- **Task 3: Refer to the course:** AZ104: Lesson 04
- **Task 4: Refer to the course:** Containerization with Docker: Lesson 02

COURSE-END PROJECT

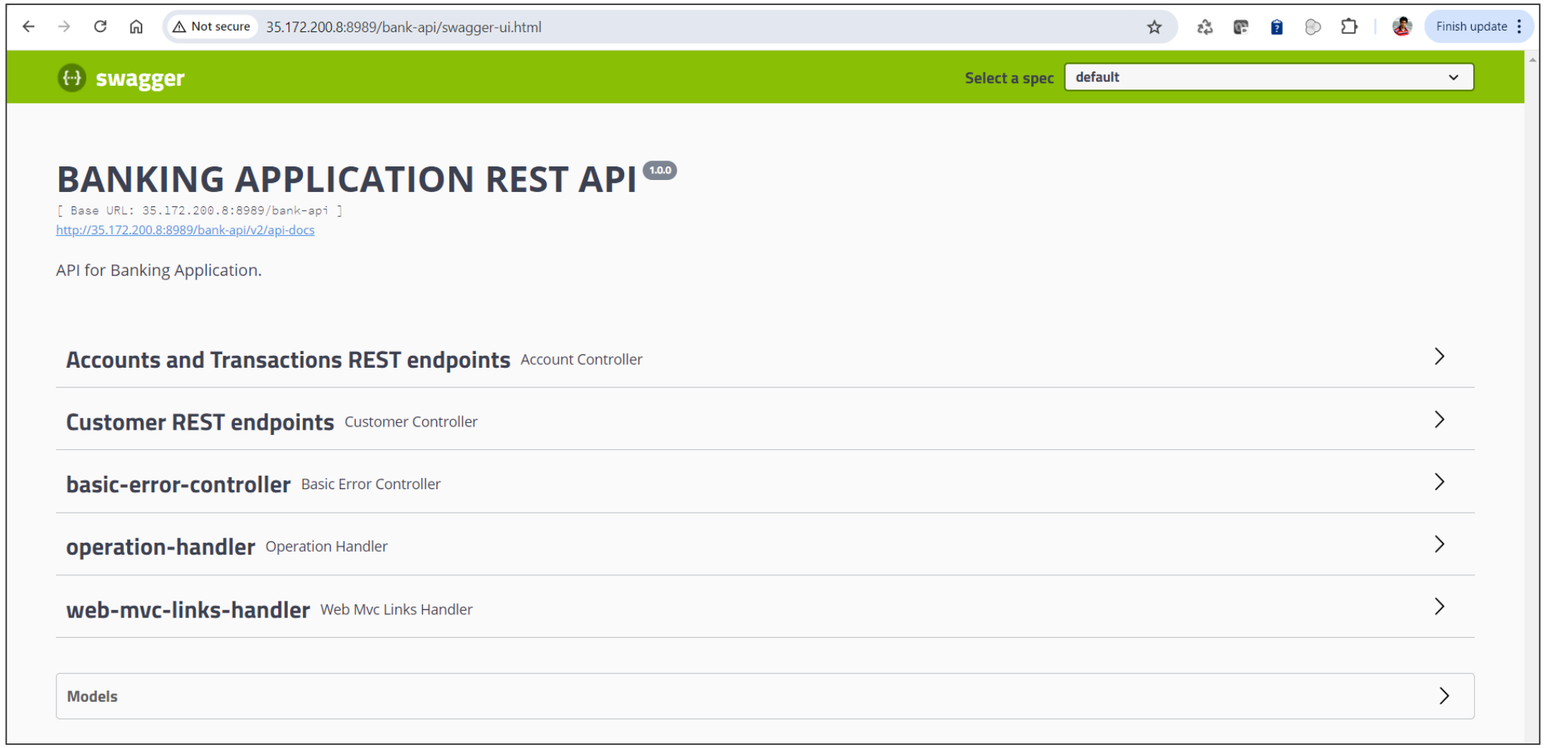
Output Screenshots

Interface of Docker image:



Output Screenshots

Banking Application.



Thank you